



Basic-mod1

basic-mod1

Medium

Cryptography

picoCTF 2022

AUTHOR: WILL HONG

Description

We found this weird message being passed around on the servers, we think we have a working decryption scheme.

Download the message [here](#).

Take each number mod 37 and map it to the following character set: 0-25 is the alphabet (uppercase), 26-35 are the decimal digits, and 36 is an underscore.

Wrap your decrypted message in the picoCTF flag format (i.e. `picoCTF{decrypted_message}`)

Hints ?

1

2

`mod 37` means modulo 37. It gives the remainder of a number after being divided by 37.

24,819 users solved

77% Liked

picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Basic-mod1

Category: Cryptography

Date: 03/22/26



I. Objective

The objective of this challenge is to understand how **modular arithmetic (mod)** can be used to encode and decode messages.

The goal is to take encoded numerical values, apply **modulo 37**, and convert the results into readable characters to reveal the hidden flag.

II. Background

In cryptography, numbers are often used to represent letters and symbols. One common technique is **modular arithmetic**, where numbers are reduced within a fixed range using a modulus.

In this challenge:

- The modulus used is **37**
- This means all numbers are mapped within the range **0–36**

Each number corresponds to a character such as:

- Letters (a–z)
- Numbers (0–9)
- Special characters (like _)

To decode the message:

1. Take each number
2. Apply **mod 37**
3. Convert the result into its corresponding character

This is a basic introduction to how encoding and decoding works in **cryptography**.

III. Tool Used

- **Kali Linux (Oracle VirtualBox)** - Used as the working environment
- **Sublime Text (subl)** - Used to view and edit files
- **Python 3** - Used to automate decoding of the message



IV. Methodology

1. Launched **Kali Linux**.
2. Downloaded the provided file using:
wget <link_address>

```
(kali@kali)~[~/Downloads]
$ wget https://artifacts.picocf.net/c/129/message.txt
--2026-03-22 08:07:54-- https://artifacts.picocf.net/c/129/message.txt
Resolving artifacts.picocf.net (artifacts.picocf.net)... 10.172.21.43, 10.172.21.26, 10.172.21.12, ...
Connecting to artifacts.picocf.net (artifacts.picocf.net)|10.172.21.43|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 86 [application/octet-stream]
Saving to: 'message.txt'

message.txt                               100%[=====] 86 --.-KB/s  in 0s
2026-03-22 08:07:56 (80.3 MB/s) - 'message.txt' saved [86/86]
```

3. Opened the encoded message using:
subl message.txt

```
(kali@kali)~[~/Downloads]
$ cd Downloads

(kali@kali)~[~/Downloads]
$ wget https://artifacts.picocf.net/c/129/message.txt
--2026-03-22 08:07:54-- https://artifacts.picocf.net/c/129/message.txt
Resolving artifacts.picocf.net (artifacts.picocf.net)... 10.172.21.43, 10.172.21.26, 10.172.21.12, ...
Connecting to artifacts.picocf.net (artifacts.picocf.net)|10.172.21.43|:443... c
HTTP request sent, awaiting response... 200 OK
Length: 86 [application/octet-stream]
Saving to: 'message.txt'

message.txt                               100%[=====] 86 --.-KB/s  in 0s
2026-03-22 08:07:56 (80.3 MB/s) - 'message.txt' saved [86/86]

(kali@kali)~[~/Downloads]
$ ls
0x41haz-164033552346.0x41haz logs.png logs.txt message.txt program-redacted.c

(kali@kali)~[~/Downloads]
$ subl message.txt

(kali@kali)~[~/Downloads]
$
```

4. Created a Python script to decode the message:
subl <filename>.py

```
(kali@kali)~[~/Downloads]
$ cd Downloads

(kali@kali)~[~/Downloads]
$ wget https://artifacts.picocf.net/c/129/message.txt
--2026-03-22 08:07:54-- https://artifacts.picocf.net/c/129/message.txt
Resolving artifacts.picocf.net (artifacts.picocf.net)... 10.172.21.43, 10.172.21.26, 10.172.21.12, ...
Connecting to artifacts.picocf.net (artifacts.picocf.net)|10.172.21.43|:443... c
HTTP request sent, awaiting response... 200 OK
Length: 86 [application/octet-stream]
Saving to: 'message.txt'

message.txt                               100%[=====] 86 --.-KB/s  in 0s
2026-03-22 08:07:56 (80.3 MB/s) - 'message.txt' saved [86/86]

(kali@kali)~[~/Downloads]
$ ls
0x41haz-164033552346.0x41haz logs.png logs.txt message.txt program-redacted.c

(kali@kali)~[~/Downloads]
$ subl message.txt

(kali@kali)~[~/Downloads]
$ subl sachigie.py

(kali@kali)~[~/Downloads]
$
```



I like naming my file with the things I like/love.

5. Wrote a Python program that:
Reads the encoded numbers
Applies **modulo 37**
Converts each result into its corresponding character

```
File Edit Selection Find View Goto Tools Project Preferences Help
message.txt x sachiqtie.py
1  #!/usr/bin/env python3
2
3  import string
4
5  flag = []
6
7  with open("message.txt") as SachiFile:
8
9      newContent = SachiFile.read()
10     numbers = [int(value) for value in newContent.split()]
11     for indexNumber in numbers:
12         modulus = indexNumber % 37
13
14         if modulus in range(26):
15             flag.append(string.ascii_uppercase[modulus])
16         elif modulus in range(26, 36):
17             flag.append(string.digits[modulus - 26])
18         else:
19             flag.append("_")
20
21 print("picoCTF{" + "".join(flag) + "}")
```

6. Ran the program (in Sublime: **Ctrl + B**) or using:
python3 <filename>.py



```
File Edit Selection Find View Goto Tools Project Preferences Help
message.txt x sachiqtie.py x
1  #!/usr/bin/env python3
2
3  import string
4
5  flag = []
6
7  with open("message.txt") as SachiFile:
8
9      newContent = SachiFile.read()
10     numbers = [int(value) for value in newContent.split()]
11     for indexNumber in numbers:
12         modulus = indexNumber % 37
13
14         if modulus in range(26):
15             flag.append(string.ascii_uppercase[modulus])
16         elif modulus in range(26, 36):
17             flag.append(string.digits[modulus - 26])
18         else:
19             flag.append("_")
20
21     print("picoCTF{" + "".join(flag) + "}")

picoCTF{R0UND_N_R0UND_ADD17EC2}
[Finished in 30ms]
```

I used CTRL + B in my case.

7. The decoded output revealed the **flag**.

```
picoCTF{R0UND_N_R0UND_ADD17EC2}
[Finished in 30ms]
```

V. Result

picoCTF{R0UND_N_R0UND_ADD17EC2}



```
picoCTF{ROUND_N_ROUND_ADD17EC2}  
[Finished in 30ms]
```

VI. Explanation

Let's analyze this script:

```
File Edit Selection Find View Goto Tools Project Preferences Help  
message.txt x sachiqtie.py  
1 #!/usr/bin/env python3  
2  
3 import string  
4  
5 flag = []  
6  
7 with open("message.txt") as SachiFile:  
8  
9     newContent = SachiFile.read()  
10     numbers = [int(value) for value in newContent.split()]  
11     for indexNumber in numbers:  
12         modulus = indexNumber % 37  
13  
14         if modulus in range(26):  
15             flag.append(string.ascii_uppercase[modulus])  
16         elif modulus in range(26, 36):  
17             flag.append(string.digits[modulus - 26])  
18         else:  
19             flag.append("_")  
20  
21 print("picoCTF{" + "".join(flag) + "}")
```

Let's try to explain it line by line...

1. `#!/usr/bin/env python3`

This line is called a **shebang**

So... What does `#!` mean?

- “#” is normally a comment in Python
- **BUT** when it's at the very first line with “!”, it becomes special

It tells the operating system (Linux):

“Hey, use this program to run this file”



So... breaking it down:
#!

Means:

“This file should be executed using the following program”

2./usr/bin/env

This is a **program in Linux**.

Its job:

“Find where a command/program is installed”

Think of it like:

A **search tool** that looks for programs in your system

3.python3

This is the program we want to use.

So the full line means:

“Use env to find python3, then run this script using it”

Why not just write this?

#!/usr/bin/python3

That would also work, BUT:

- Not all systems have Python in /usr/bin/python3
- Some systems install Python somewhere else

So using:

#!/usr/bin/env python3

makes your script **portable** (works on more systems)



Why is this important?

Without shebang, you must run:

Python3 decode.py

With shebang + executable permission:

Chmod +x decode.py

./decode.py

4. **import string**

Imports a built-in Python module.

Gives access to:

- `string.ascii_uppercase` → ABCDEFGHIJKLMNOPQRSTUVWXYZ
- `string.digits` → 0123456789

5. **flag = []**

Creates an **empty list**.

This will store decoded characters one by one.

6. **with open("message.txt") as SachiFile:**

Opens the file safely.

- "message.txt" → the file
- SachiFile → variable name

with ensures:

- File is **automatically closed** after use

7. **newContent = SachiFile.read()**

Reads the whole file as text.

8. **numbers = [int(value) for value in newContent.split()]**



This is a **list comprehension** (shortcut loop).

Step-by-step:

- `.split()` → splits text by spaces
- `int(value)` → converts each into a number

result:

[1, 2, 3, 4, 5]

9. **for indexNumber in numbers:**

Loops through each number.

Like:

- first → 54
- next → 12

10. **modulus = indexNumber % 37**

Applies **modulo 37**

Keeps values between:

0 → 36

Example:

- $54 \% 37 = 17$
- $99 \% 37 = 25$

11. **if modulus in range(26):**

Checks if value is:

0 → 25

These represent **letters A–Z**

12. **flag.append(string.ascii_uppercase[modulus])**



Adds a letter to the list.

Example:

- $0 \rightarrow A$
- $1 \rightarrow B$

13. `elif modulus in range(26, 36):`

Checks if value is:

$26 \rightarrow 35$

These represent digits (0–9)

14. `flag.append(string.digits[modulus - 26])`

Why - 26?

Because:

- 26 should map to digit 0 (based in description)
- $27 \rightarrow 1$

Fixes the index.

15. `else:`

Handles the last value:

36

16. `flag.append("_")`

If value = 36 $\rightarrow$ underscore _

17. `print("picoCTF{" + "".join(flag) + "}")`

Final output:

- `"".join(flag)` $\rightarrow$ combines letters into a word
- Adds picoCTF {} format



That should wrap things up in the script... Is it understandable?

So...

What is “mod” (modulo)?

“Mod” means **remainder after division**.

Example:

- $10 \bmod 3 = 1$ (because $10 \div 3$ leaves remainder 1)

Why mod 37?

Idk? Maybe because there are **37 possible characters**:

- 26 letters (A–Z)
- 10 digits (0–9)
- 1 special character (_)

So every number gets mapped into this range using mod 37 ig.

How does decoding work?

Think of it like a **code wheel**:

- Each number points to a character
- If the number is too big, we “wrap it around” using mod

Example:

- $40 \bmod 37 = 3$
- So we use character at index **3**

Why use Python?

Instead of decoding manually (which is slow and error-prone), Python:



- Automates the process
- Quickly converts all numbers into characters
- Ensures accuracy

VII. Conclusion

This challenge demonstrates how **modular arithmetic is used in cryptography to encode and decode messages.**

By applying **mod 37** and mapping numbers to characters, the hidden message was successfully decoded. Using Python made the process efficient and reliable.

This reinforces an important concept in cybersecurity:

Mathematics is a fundamental tool in encryption and decryption.

— The Analyst: Hyposelenia